

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

*I **GAYATHRI N P (192421362)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

Problem:

The perceptron is trained on a linearly separable dataset but does not converge.

Possible Causes:

- Learning rate is too high or too low.
- Incorrect weight initialization.
- Bias value is missing or incorrectly updated.
- Error in the perceptron weight update rule.
- Incorrect implementation of the activation function.
- Errors or inconsistencies in the training data.

Debugging Steps:

1. Verify that the dataset is truly linearly separable.
2. Check the perceptron update rule:
$$\text{Weight} = \text{Weight} + \text{Learning Rate} \times \text{Error} \times \text{Input}.$$
3. Ensure the bias is included and updated correctly.
4. Monitor weight values after each iteration.
5. Test the algorithm with a small sample dataset.
6. Check that the stopping condition is correctly implemented.

Optimization Techniques:

- Choose an appropriate learning rate.
- Initialize weights with small random values.
- Normalize input features.
- Shuffle the training data before each epoch.
- Use early stopping when convergence is achieved.

Conclusion:

Correct implementation of the update rule, proper learning rate selection, and normalization help the perceptron converge faster and more reliably.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

Problem:

A multilayer perceptron shows slow convergence and gets stuck in local minima.

Possible Causes:

- Vanishing gradients.
- Poor weight initialization.
- Inappropriate activation functions.
- Learning rate is too high or too low.
- Unnormalized input data.

Debugging Steps:

1. Monitor training and validation loss.
2. Check gradient values during backpropagation.
3. Verify weight updates after every iteration.
4. Test different activation functions.
5. Ensure the data is normalized.
6. Experiment with different learning rates.

Optimization Techniques:

- Use ReLU activation instead of Sigmoid where appropriate.
- Apply Xavier or He weight initialization.
- Normalize the input data.
- Use adaptive optimizers such as Adam or RMSProp.
- Apply learning rate scheduling.
- Use Batch Normalization.

Conclusion:

Adaptive optimizers, proper initialization, and normalization improve convergence speed and help avoid local minima.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

Problem:

A genetic algorithm produces inconsistent and suboptimal solutions.

Possible Causes:

- Incorrect chromosome encoding.
- Poorly designed fitness function.
- Premature convergence.
- Low population diversity.
- Mutation rate is too low or too high.

Debugging Steps:

1. Verify chromosome representation.
2. Check whether the fitness function reflects the optimization goal.
3. Monitor fitness values across generations.
4. Observe crossover and mutation operations.
5. Check population diversity after each generation.

Optimization Techniques:

- Increase mutation rate when diversity is low.
- Adjust selection pressure using tournament or roulette wheel selection.
- Apply elitism to preserve the best solutions.
- Increase population size.
- Use adaptive crossover and mutation probabilities.

Conclusion:

Balanced mutation, effective selection, and elitism improve solution quality and speed up convergence.

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

Problem:

The model suffers from overfitting and high computational cost.

Possible Causes:

- Very complex neural network.
- Too many parameters.
- Small training dataset.
- Long training time.
- Lack of regularization.

Debugging Steps:

1. Compare training and validation accuracy.
2. Analyze model size and number of parameters.
3. Check for unnecessary hidden layers.
4. Evaluate training time and memory usage.
5. Verify data quality and quantity.

Optimization Techniques:

- Apply L1 or L2 Regularization.
- Use Dropout layers.
- Prune unnecessary neurons and connections.
- Apply Early Stopping.
- Use efficient evaluation methods.
- Increase training data or use data augmentation.

Conclusion:

Regularization, dropout, pruning, and efficient model design reduce overfitting while improving computational efficiency.

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

Problem:

The model suffers from overfitting and high computational cost.

Possible Causes:

- Very complex neural network.
- Too many parameters.
- Small training dataset.
- Long training time.
- Lack of regularization.

Debugging Steps:

1. Compare training and validation accuracy.
2. Analyze model size and number of parameters.
3. Check for unnecessary hidden layers.
4. Evaluate training time and memory usage.
5. Verify data quality and quantity.

Optimization Techniques:

- Apply L1 or L2 Regularization.
- Use Dropout layers.
- Prune unnecessary neurons and connections.
- Apply Early Stopping.
- Use efficient evaluation methods.
- Increase training data or use data augmentation.

Conclusion:

Regularization, dropout, pruning, and efficient model design reduce overfitting while improving computational efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand